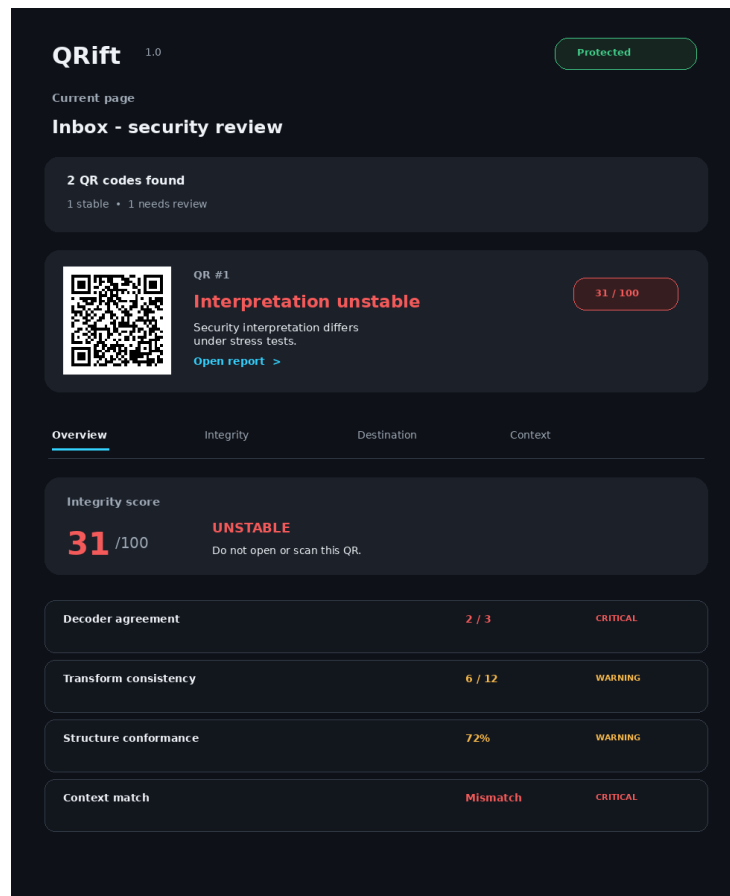


QRift

QR Integrity Firewall for Chrome

MASTER PRODUCT REQUIREMENTS DOCUMENT



Product direction: a security-oriented Chrome side panel, not a generic QR scanner.

Core promise

QRift does not ask only "what does this QR decode to?" It asks "does this QR have one stable, trustworthy interpretation across decoding and realistic rendering conditions?"

Product	QRift
Platform	Chrome Extension, Manifest V3
Primary wedge	Gmail / webmail QR verification
Document status	Build-ready master specification, v1.0

0. Executive Build Brief

QRift is a Chrome security extension that detects QR codes in Gmail and other webpages, verifies their structural integrity, compares independent decoding paths, stress-tests rendering conditions, checks the destination and page context, and produces an explainable verdict. The product is intentionally designed as a browser-side integrity layer rather than another generic URL reputation scanner.

Product differentiation

Existing browser and email-security products already scan QR codes, decode URLs, and apply phishing/reputation analysis. QRift differentiates by testing the QR artifact itself for interpretation instability - the class of problem highlighted by the assigned dual-module QR research. This is a stronger and narrower product thesis than "AI QR phishing detection."

- **Primary user:** security-conscious employees and analysts reviewing QR-bearing email or web content.
- **Primary workflow:** detect -> quick verify -> deep verify when needed -> compare interpretations -> block/warn -> report.
- **Core signal:** Interpretation Stability / Drift, combining decoder agreement, metamorphic stability, and structural conformance.
- **AI role:** optional supporting anomaly classifier; never the only security decision path.
- **MVP architecture:** WXT + React + TypeScript, Manifest V3, content script, service worker, side panel, offscreen processing, browser-local QR engines, optional API for cloud reputation.
- **Privacy posture:** local-first. Raw QR images remain on-device unless the user/admin explicitly enables cloud analysis.
- **Competition demo:** show a conventional QR that is stable, then a controlled dual-module fixture that produces divergent interpretations under selected test conditions.

Critical product constraint

The paper does not establish that every dual-module QR will decode differently under every decoder or transformation. QRift must therefore report observed instability as evidence, not claim universal detection. Likewise, "simulate phone" is a rendering approximation unless calibrated against a real device.

1. Problem Definition

QR codes remove the human-readable part of a link. The assigned paper argues that this creates a security gap because a QR detection engine can identify and decode an image while still missing structural manipulation that causes a security tool and a user device to interpret the same image differently. The paper describes a dual-module construction in which one QR layer is overlaid by a second QR and reports a successful bypass rate of 99.87% +/- 0.1277% under the tested Sophos setup, across 2,592 successful test emails with zero failures. This statistic is specific to that experiment and should not be presented as a universal bypass rate. [PAPER: pp. 13-16]

The product problem is therefore broader than "malicious URL detection": security systems need a way to determine whether a QR artifact is internally consistent and whether its interpretation remains stable under plausible decoding and rendering conditions.

2. Research-to-Product Translation

Paper observation	Product requirement	Why it matters
QRift Master PRD v1.0 Chrome Extension 2		

Simple/predictable QR decoding can be exploited. [PAPER: pp. 9-10]	Use multiple independent decoding strategies plus stress testing.	Removes dependence on a single decoding path.
Dual-module QR can embed a benign QR over a malicious base QR. [PAPER: pp. 13-16]	Inspect per-module statistics and look for embedded substructure.	Adds artifact-level evidence even when one decoder returns a benign payload.
Rendering size can hide submodules from users while preserving raw pixels. [PAPER: p. 16]	Run rendered-size and resampling tests.	Tests the gap between stored image and user-visible image.
Camera apps may behave differently by distance and size. [PAPER: pp. 16-17]	Use context simulation / controlled render transformations.	Models the instability mechanism without claiming exact device emulation.
Defense recommendations include structure, AI visual analysis, and context simulation. [PAPER: pp. 19-22]	Build three signal families: structure, behavior under transforms, and context.	Directly turns the paper recommendations into product modules.
Open-source decoders were susceptible in the study. [PAPER: pp. 16-17]	Adapter-based decoder diversity with independent implementations.	Makes decoder disagreement a first-class signal.

3. Product Positioning

One-liner

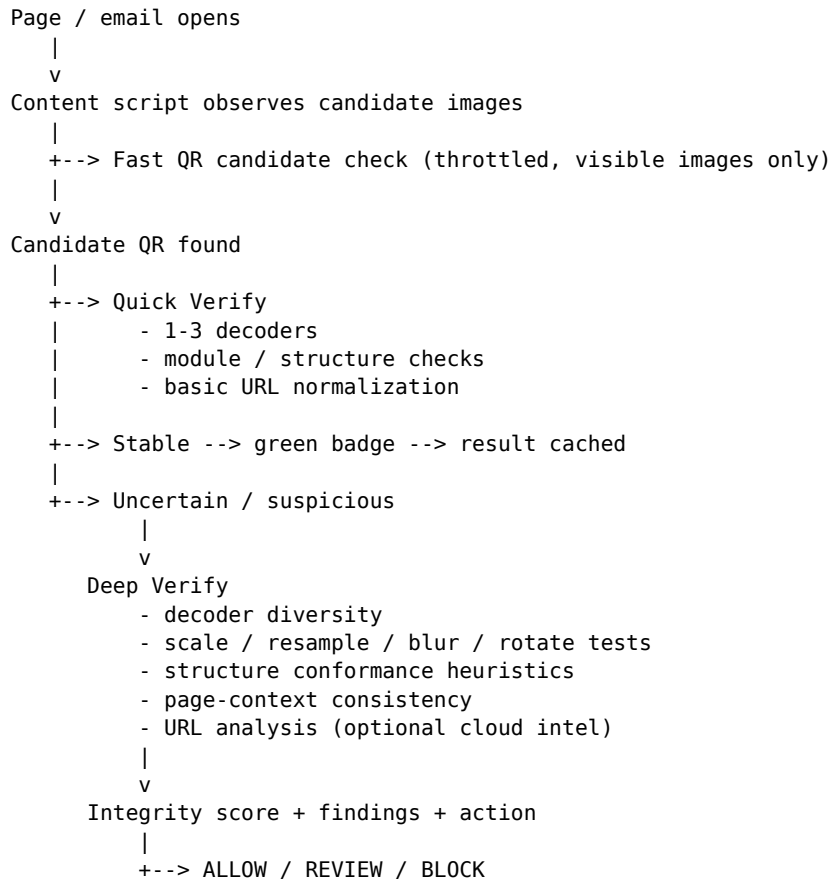
"QRift verifies that a QR has one trustworthy meaning before you trust it."

What others optimize	What QRift optimizes
Decode a QR quickly	Verify interpretation stability
Determine whether destination is malicious	Determine whether the QR artifact itself is internally trustworthy
Known-bad URL / reputation	Unknown-manipulation detection signals
Single decoding path	Decoder diversity + transform diversity
Passive scan	Active stress test
Black-box risk score	Evidence matrix + explainable findings

4. Users and Jobs-to-be-Done

Persona	Job	Success state
Employee / student	"Tell me whether I should scan this QR in the email."	Answer is understandable in under 5 seconds.
Security analyst	"Show me why the QR is suspicious."	Evidence includes decoder results, transforms, structure, URL, context.
Security admin	"Control policy without reading everyone's email."	Scoped configuration, privacy controls, audit events.
Researcher / red teamer	"Reproduce and study the weakness safely."	Controlled test fixture lab; no live malicious infrastructure required.

5. End-to-End Product Working



The extension should avoid running expensive analysis on every image. It uses a candidate queue, visibility heuristics, caching, and escalation: fast checks first, deep checks only when needed or explicitly requested.

6. UI / UX Design System

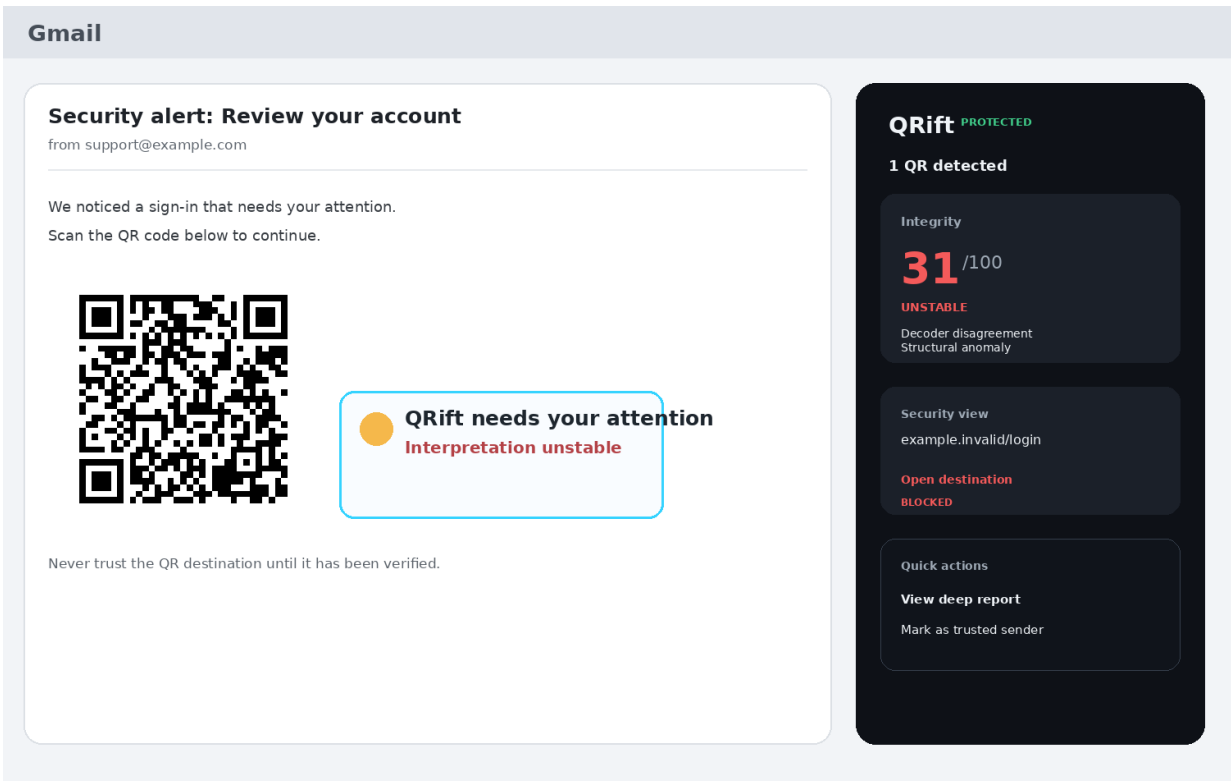
The interface should look like a serious browser security product: restrained, information-dense, readable, and explainable. Avoid the visual language of generic AI products: no rainbow gradients, glassmorphism, oversized marketing typography, or excessive pill badges.

Token	Direction
Primary surface	Deep graphite; high-contrast light text.
Secondary surface	Slate cards with 1 px borders.
Accent	Electric cyan used sparingly for links, focus, and active tabs.
Good	Green used only for stable / trusted states.
Review	Amber used for uncertainty or partial evidence.
Critical	Red used for block-level findings only.
Typography	Inter or system sans; tabular numerals for score and metrics.
Radius	14-18 px for cards; 10-12 px controls; avoid "everything is a pill."
Motion	Fast 120-180 ms transitions; no looping animations except scan progress.

Accessibility	Keyboard reachable, visible focus, labels on icon-only controls, minimum readable contrast.
---------------	---

7. Frontend Information Architecture

Surface	Purpose	Primary components
Inline Gmail overlay	Give immediate signal without hijacking email.	QR status badge, risk tooltip, "Open report" action.
Chrome side panel	Primary product workspace.	Current page summary, QR list, score, findings, tabs.
Deep verification view	Analyst-grade evidence.	Interpretation comparison, evidence matrix, transform outcomes, destination.
Settings	Privacy + scope + policy.	Scanning mode, cloud intelligence, history, trusted senders.
Research Lab	Controlled demo/test workspace.	Fixture selector, decoder matrix, transform runner, JSON export.



Gmail inline concept: the extension does not cover the email; it attaches a small, high-signal status layer to the QR.

QRift / Deep Verification

Explainable evidence - not a black-box verdict

VERDICT

BLOCK

Interpretation instability detected

31 / 100

Security decoder

https://example.com/help

Observed under different decoding conditions.

Payload mismatch

Alternate decoder

https://example.invalid/login

Observed under different decoding conditions.

Payload mismatch

Evidence matrix

Test	Condition	Result	Signal
Decoder A	native	example.com	stable
Decoder B	center sample	example.com	stable
Decoder C	alternate path	example.invalid	diverged
Scale 50%	rendered	example.invalid	changed
Blur 1.5px	rendered	example.com	flipped

Action: Do not open the destination. Report or quarantine the message.

Deep verification concept: the decisive UI is evidence, not a mysterious risk number.

QRift Master PRD v1.0 | Chrome Extension | 6

QRift settings

Control privacy, scanning scope, and analysis depth.

Protection mode

Run verification only when a QR is detected



Gmail inline protection

Show a small status badge beside QR codes



Auto deep verification

Only run after a quick scan looks unusual



Cloud URL intelligence

Send hashed destination metadata for reputation checks



Store raw QR images

Disabled by default



Privacy principle

Images stay on-device unless you explicitly enable cloud analysis.

Settings concept: local-first privacy is visible and understandable.

8. Side Panel Detailed Specification

- **Header:** QRift wordmark, product status, settings icon, and current page title.
- **Page summary card:** number of QR codes found; counts by stable/review/critical.
- **QR list:** thumbnail, QR index, short verdict, score, report action.
- **Overview tab:** score, recommendation, top evidence signals, user action.
- **Integrity tab:** structure findings + module statistics + anomaly map placeholder.
- **Destination tab:** decoded URL(s), normalized URL, hostname, redirect preview, reputation status.
- **Context tab:** email/page claim vs QR destination; sender/domain relationship; mismatch explanation.
- **Footer:** analysis time, privacy mode, "report a false positive", cache status.

9. Frontend Component Contract

```
src/ui/  
  components/  
    AppShell.tsx  
    Header.tsx  
    PageSummary.tsx  
    QRCard.tsx  
    QRThumbnail.tsx  
    IntegrityScore.tsx  
    VerdictBanner.tsx
```

```

EvidenceRow.tsx
DecoderTable.tsx
TransformMatrix.tsx
DestinationPanel.tsx
ContextPanel.tsx
ActionBar.tsx
PrivacyBadge.tsx
EmptyState.tsx
ErrorState.tsx
screens/
  SidePanelHome.tsx
  QRDetail.tsx
  DeepVerify.tsx
  Settings.tsx
  ResearchLab.tsx
lib/
  formatters.ts
  scorePresentation.ts
  validators.ts

```

Component	Props / data	Behavior
IntegrityScore	score:number, label, confidence	Animated count-up only on first mount; accessible text mirror.
QRCard	qr:QRArtifact, onOpen	Entire row keyboard clickable; danger states do not rely on color alone.
DecoderTable	results:DecodeResult[]	Groups equal payloads; clearly separates "failed to decode" from "decoded differently".
TransformMatrix	tests:TransformResult[]	Each row shows condition, payload hash, decoder count, and changed/not changed.
VerdictBanner	verdict, reasons[]	Always explains top 1-3 reasons and action.
ActionBar	actions[], disabled reasons	Block/open/report/copy actions; destructive actions require confirm where appropriate.

10. Interaction Rules

- 1. No forced takeover:** QRift must not automatically navigate the user to decoded destinations.
- 2. Progressive disclosure:** Most users see verdict + one sentence; analysts can expand evidence.
- 3. No score without reason:** Every score must have at least one human-readable driver.
- 4. No "verified" unless scope is explicit:** "Stable" means stable under the tests QRift actually ran.
- 5. Copy safe destination:** For suspicious QR, copying the decoded URL is allowed only as inert text; never auto-open it.
- 6. Manual deep verify:** Available on any QR even when quick verification is stable.
- 7. False positive feedback:** User can flag a QR as safe; this should update a local allowlist, not silently override all policy.
- 8. Trusted sender is not trusted QR:** Sender trust and QR integrity are separate signals.

11. Technical Architecture

Target implementation is a Chrome Manifest V3 extension. Chrome documents the Side Panel API for browser-side UI and the Offscreen API for hidden document contexts. Permissions should use least privilege; use `activeTab` for user-initiated operations and optional host permissions for broader scanning. [WEB: Chrome Extensions API/Permissions docs, accessed 2026-09-11]

```
QRift monorepo

/apps/extension
  entrypoints/
    background.ts          # MV3 service worker
    content.ts             # page/Gmail integration
    sidepanel.tsx          # main side panel UI
    popup.tsx              # optional compact entry point
    offscreen.html / offscreen.ts
  public/
  src/
    core/
    decoders/
    transforms/
    scoring/
    storage/
    privacy/
    ui/
  wxt.config.ts

/apps/api (optional)
  src/
    routes/
      analyze.ts
      reputation.ts
      health.ts
    services/
      reputation.ts
      hashing.ts
      policy.ts

/packages/shared
  schemas.ts
  constants.ts
  types.ts
  fixtures.ts
```

12. Chrome Extension Permissions

Permission / capability	Use	MVP decision
sidePanel	Host QRift workspace beside the page.	Required.
storage	Persist settings, cache, local scan metadata.	Required.
scripting	User-invoked deep scan / controlled injection.	Use only when necessary.
activeTab	Temporary user-gesture access for current tab.	Preferred for manual scan.
contextMenus	"Verify QR with QRift" on images / selection.	Optional but useful.
host permission: mail.google.com	Gmail inline detection.	Required for Gmail mode.
optional host permissions for web	Broader automatic web protection.	Request only when user enables web-wide mode.

declarativeNetRequest	Network blocking rules.	Do not require for MVP; QRift can warn/block inside its own UI.
-----------------------	-------------------------	---

Permission principle

Do not request all-web access just to make the demo work. A strong security product should minimize its own attack surface and permission footprint.

13. Core Engine Modules

Module	Input	Output	Implementation
QR Candidate Detector	Image element / cropped screenshot	candidate region + confidence	jsQR/ZXing fast probe + square/aspect heuristics
Decoder Adapter Layer	QR bitmap	DecodeResult[]	Adapters for ZXing, jsQR, OpenCV QRCodeDetector; isolate libraries behind a common interface.
Structural Forensics	Normalized QR bitmap	StructureReport	module grid estimation, average-vs-center contrast, finder/alignment consistency, noise/substructure flags
Metamorphic Engine	QR bitmap	TransformResult[]	scale, resample, blur, JPEG, rotation, perspective, contrast; decode each variant
Context Analyzer	page/email text + URL	ContextReport	brand/claim mismatch, sender/domain mismatch, urgency cues; deterministic heuristics first
URL Analyzer	decoded URL	DestinationReport	normalize, hostname, redirects when available, URL syntax, optional reputation
Scoring Engine	all reports	IntegrityScore + findings	weighted, explainable, calibrated thresholds
Policy Engine	score + findings + user policy	allow/review/block	rule evaluation with explicit overrides
Evidence Store	scan result	local history	hashes + metadata; raw image retention off by default

14. Decoder Interface

```
export type DecodeResult = {
  decoderId: string;
  success: boolean;
  payload?: string;
  payloadType?: 'url' | 'text' | 'wifi' | 'other';
  confidence?: number;
  elapsedMs: number;
  error?: string;
};

export interface QRDecoder {
  id: string;
  decode(image: ImageBitmap | ImageData): Promise<DecodeResult>;
}
```

```

}

export interface DecoderRegistry {
  list(): QRDecoder[];
  runAll(input: ImageBitmap): Promise<DecodeResult[]>;
}

```

15. Structural Forensics Specification

The paper recommends moving beyond decode-and-analyze and specifically suggests comparing the average color of a module with its center pixel, because a dual-module may change the center or internal pixels without looking suspicious as a whole. [PAPER: pp. 19-20]

- 1. Locate QR geometry:** obtain corner/finder estimates and perspective-correct the image.
- 2. Estimate version/module grid:** derive candidate module count; if uncertain, keep a confidence value rather than inventing a version.
- 3. Partition module cells:** for each module, compute mean luminance, center luminance, variance, and edge density.
- 4. Detect internal contrast:** flag modules where center-vs-mean difference is above an adaptive threshold.
- 5. Check repeated pattern:** cluster anomalous modules to see whether they form coherent submodule-like geometry.
- 6. Check standards-facing structure:** finder patterns, quiet zone, timing patterns, format information, alignment patterns; use conformance as an evidence signal, not a binary gate.
- 7. Produce heatmap:** render a semi-transparent anomaly layer aligned with the QR image.

Important

Do not implement "center pixel differs = malicious". It is a signal. Logos, antialiasing, compression, color QR codes, and display artifacts can also create within-module variation. Thresholds must be calibrated against benign QR variants.

16. Metamorphic / Stress Test Matrix

Transform	Purpose	Default variants	Risk signal
Scale	Expose size-dependent decoding	1.0x, 0.75x, 0.5x, 0.33x	Payload flip / decoder dropout
Resample	Test interpolation sensitivity	nearest, bilinear, bicubic	Payload flip
Blur	Approximate focus loss	0.5, 1.0, 1.5 px	Payload flip / unstable decode
Rotation	Test orientation robustness	-10, -5, +5, +10 deg	Payload flip / dropout
Perspective	Approximate camera angle	mild corner warps	Payload flip
JPEG	Approximate compressed delivery	quality 95, 80, 60	Payload flip / dropout
Contrast / brightness	Test rendering variability	small +/- adjustments	Unexpected payload change
Print-scan approximation	Future / optional	dither + blur + perspective + noise	Divergent decode

17. Interpretation Stability Score

The score is an MVP heuristic, not a research-validated security metric. The code should keep the raw component metrics and reasons so weights can later be learned or recalibrated.

```
payloadConsensus      = agreement among successful decoders
transformConsistency = fraction of stress tests preserving the reference payload
structureConformance  = 1 - normalized structural anomaly score
contextConsistency    = normalized match between page claim / sender / destination
```

MVP weighted score (0-100):

```
stability =
    0.40 * payloadConsensus +
    0.30 * transformConsistency +
    0.20 * structureConformance +
    0.10 * contextConsistency
```

```
score = round(clamp(stability, 0, 1) * 100)
```

MVP presentation thresholds:

```
85-100  STABLE
70-84   REVIEW
40-69   UNSTABLE
0-39    CRITICAL
```

Hard-stop policy examples:

- confirmed decoder payload mismatch -> CRITICAL / BLOCK
- repeated transform payload flip -> at least REVIEW
- severe structural anomaly + suspicious destination -> CRITICAL

Calibration requirement

Do not claim the thresholds are scientifically optimal. Build a labeled fixture set, measure false-positive rate, and tune the weights/thresholds empirically before presenting accuracy claims.

18. Destination Analysis

QRift should not compete head-on with dedicated URL reputation vendors. Its destination layer is a supporting control that explains what the QR points to and whether obvious risk signals are present.

- **Normalize URL:** scheme, hostname, port, punycode, percent encoding, fragments, default ports.
- **Show hostname first:** the hostname is the critical human-readable security fact.
- **Redirect chain:** optional remote resolution; never auto-visit on the user device in a hidden context.
- **URL heuristics:** IP-literal hosts, suspicious encodings, excessive subdomains, uncommon ports, lookalike hostnames.
- **Reputation adapter:** cloud endpoint can query a provider such as Google Safe Browsing or another approved service; keys must never be shipped in the extension bundle.
- **Decision separation:** a clean URL does not make an unstable QR safe; a malicious URL does not explain structural instability.

19. Page Context Analysis

Context is a deliberate product extension of the paper's recommendation to analyze the QR in the context in which it is presented. The first implementation should be deterministic and explainable.

Signal	Example	UI wording
Brand mismatch	Email says Microsoft, QR points to unrelated domain	"QR destination does not match the claimed brand."
Sender/destination mismatch	sender domain differs from destination organization	"Sender and destination are unrelated."
Urgency cue	"Verify now" / "account will be disabled"	"Message uses urgency to drive a QR action."
Action ambiguity	QR destination differs from visible call-to-action link	"QR and visible link point to different destinations."

20. Data Model

```

type QRArtifact = {
  id: string;
  source: 'gmail' | 'webpage' | 'context-menu' | 'lab';
  pageUrl?: string;
  imageHash: string;
  thumbnailDataUrl?: string; // only when local retention is enabled
  detectedAt: string;
  bbox?: { x:number; y:number; width:number; height:number };
};

type ScanReport = {
  scanId: string;
  artifactId: string;
  status: 'stable' | 'review' | 'unstable' | 'critical' | 'error';
  score: number;
  confidence: number;
  decoders: DecodeResult[];
  structure: StructureReport;
  transforms: TransformResult[];
  destination?: DestinationReport;
  context?: ContextReport;
  findings: Finding[];
  action: 'allow' | 'review' | 'block';
  createdAt: string;
};

```

21. Storage and Privacy

Data	Default retention	Reason
Raw QR image	Not stored	Minimize sensitive content exposure.
Image hash	30 days local	Deduplicate/caching without keeping image.
Decoded URL	Local only, 30 days	Needed for history; can be disabled.
Scan metadata	30 days local	Explain previous verdicts.
Cloud reputation request logs	Provider-defined / minimal	Operational debugging; send only hashes/hostnames where possible.

Privacy mode

"Local only" must be a real mode: no image upload, no page-content upload, and no background URL

reputation calls. The UI should show a visible Local badge.

22. Optional API Contract

```
POST /v1/analyze
Content-Type: application/json
```

```
{
  "imageBase64": "...",    // only in Cloud Analysis mode
  "options": {
    "deep": true,
    "reputation": false,
    "context": {"senderDomain": "example.com", "pageText": "..."}
  }
}
```

Response 200

```
{
  "scanId": "scan_123",
  "score": 31,
  "status": "critical",
  "action": "block",
  "interpretations": [
    {"payload": "https://example.com/help", "sources": ["zxing", "jsqr"]},
    {"payload": "https://example.invalid/login", "sources": ["opencv"]}
  ],
  "findings": [
    {"code": "DECODER_MISMATCH", "severity": "critical"},
    {"code": "STRUCTURE_ANOMALY", "severity": "warning"}
  ]
}
```

23. Message Passing Contract

```
// content -> service worker
{ type: 'QR_CANDIDATES_FOUND', tabId, candidates: [{src, bbox, imageId}] }
{ type: 'START_DEEP_VERIFY', artifactId }
{ type: 'OPEN_SIDE_PANEL', artifactId }

// service worker -> offscreen
{ type: 'RUN_QR_ANALYSIS', jobId, imageData, mode: 'quick' | 'deep' }
{ type: 'RUN_TRANSFORM_BATCH', jobId, imageData, transformIds: [...] }

// service worker -> UI
{ type: 'SCAN_PROGRESS', scanId, stage, percent }
{ type: 'SCAN_COMPLETE', report }
{ type: 'SCAN_ERROR', scanId, code, message }
```

24. Gmail Integration Strategy

Gmail is a single-page application and its DOM can change. The integration therefore needs to be selector-light and event-driven rather than depending on one fragile CSS path.

1. **MutationObserver**: watch for meaningful DOM additions/removals in the message viewport, not the entire page at high frequency.

- 2. **IntersectionObserver**: only enqueue visible or near-visible images.
- 3. **Candidate queue**: dedupe by image source/hash + bounding box and process in controlled batches.
- 4. **Image acquisition**: try direct fetch via extension context where allowed; fall back to a visible-tab screenshot crop for user-initiated scans.
- 5. **Inline badge anchor**: position a small badge relative to the image container; re-anchor on layout changes.
- 6. **SPA navigation**: listen for route/view changes and clear stale page state safely.
- 7. **Failure handling**: if Gmail DOM extraction fails, the side panel still offers "Verify selected image / visible page."

Scope control

The initial extension should optimize for Gmail on desktop Chrome. Do not block the first release on perfect support for Outlook, PDF viewers, and every website. Those are integration surfaces, not the core detection engine.

25. Research Lab / Competition Demo Mode

The Research Lab is a hidden or developer-enabled screen designed specifically for the competition demo and regression testing. It makes the paper-to-prototype connection visible without shipping offensive infrastructure.

- **Fixture A - stable QR**: reserved-domain URL that decodes identically across adapters.
- **Fixture B - dual-module test**: controlled image built from benign + placeholder malicious domains such as example.invalid; no live phishing endpoint.
- **Fixture C - visual-noise QR**: compressed/resized or logo-bearing benign QR to measure false positives.
- **Fixture D - decoder dropout**: intentionally low-quality QR to separate "decode failed" from "decoded differently."

26. Test Dataset Plan

Bucket	Minimum MVP size	Purpose
Benign clean QRs	250	Baseline false-positive measurement.
Benign style variants	150	Logos, color, compression, resizing.
Dual-module controlled fixtures	100	Core attack-class signal validation.
Other QR obfuscation fixtures	100	Check whether engine generalizes without overfitting.
Broken / low-quality QRs	100	Distinguish poor scan quality from malicious instability.

Create a fixture manifest with ground-truth labels and provenance. Store only test URLs and placeholder domains in the repository. Do not include live phishing URLs in code or demo assets.

27. Evaluation Metrics

Metric	Definition	Target for MVP demo
TPR on controlled dual-module fixtures	fraction correctly flagged as	>= 90% on held-out controlled fixtures;

	unstable/critical	report actual measured value
False positive rate on benign set	benign fixtures incorrectly flagged	<= 5%; tune thresholds to achieve
Decoder agreement rate	fraction of benign QRs with same successful payload	>= 95%
Transformation consistency	fraction of benign transform tests preserving payload	>= 95%
Median quick latency	time to first verdict	< 500 ms local on reference laptop
Median deep latency	time to full report	< 2.5 s local on reference laptop

How to talk about results

Present your measured dataset, sample sizes, operating conditions, and confidence intervals. Never turn a competition prototype result into a blanket claim about all QR scanners or all dual-module attacks.

28. Automated Test Plan

- **Unit tests:** URL normalization, payload grouping, score calculation, transform naming, evidence ranking.
- **Decoder adapter tests:** known clean QR fixtures, known dual-module fixtures, malformed images.
- **Structural tests:** expected center-vs-mean anomaly behavior on synthetic module patterns.
- **Metamorphic tests:** same benign QR should preserve payload across normal transforms; unstable fixture should produce documented divergence when it does.
- **Extension tests:** message passing, side panel open, content-script candidate detection, storage isolation.
- **Playwright E2E:** load a local test page with known QR images; click verify; assert verdict and evidence.
- **Regression corpus:** every confirmed bug adds a fixture.

29. Error Handling / Resilience

Failure	User outcome	Internal behavior
QR library unavailable	"Verification incomplete" - not "safe"	Fall back to another decoder; record capability gap.
Image unreadable	"Could not verify"	Offer re-run / zoom / selected image scan.
Decoder disagreement	Show divergence	Escalate to deep verify automatically.
Network unavailable	Local-only report	Reputation badge shows unavailable, not clean.
Permission denied	Explain which action needs permission	Do not repeatedly prompt.
Gmail DOM changed	Fallback to manual scan	Telemetry only if user opted in.

30. Security Requirements for QRift Itself

1. **Least privilege:** minimum permissions; no background access to unrelated content.
2. **No active browsing of untrusted URLs:** all destination analysis should be done through safe server-side fetchers or inert parsing.
3. **No secret keys in extension:** cloud provider secrets stay on the backend.

4. **Sanitize all rendered page text:** context snippets displayed in extension UI must be treated as untrusted text.
5. **No remote code execution:** do not load arbitrary scripts from remote origins into extension pages.
6. **Immutable evidence:** store hash + timestamp with result; if cloud, sign or integrity-protect audit records.
7. **Abuse controls:** rate limit any cloud API; cap image size and transform count.
8. **Supply-chain pinning:** lock decoder package versions and verify production builds.

31. Product Rules / Guardrails

Rule 1

A single decoder returning a URL is not enough to label a QR trustworthy.

Rule 2

Decoder failure is not decoder disagreement. Keep those states distinct.

Rule 3

AI is supporting evidence. The deterministic evidence matrix is the product moat.

Rule 4

A stable QR can still point to a malicious site. Stability is not destination safety.

Rule 5

A suspicious QR does not need to decode to a second URL to deserve review; structure and transform instability can be sufficient evidence.

32. Complete Repository / File Structure

```
qrfit/  
  apps/  
    extension/  
      entrypoints/  
        background.ts  
        content.ts  
        sidepanel/  
          index.html  
          main.tsx  
        popup/  
          index.html  
          main.tsx  
        offscreen/  
          index.html  
          main.ts  
      src/  
        core/  
          pipeline.ts  
          types.ts  
          constants.ts  
        decoders/
```

```

    decoder.ts
    zxing.ts
    jsqr.ts
    opencv.ts
    registry.ts
transforms/
  scale.ts
  resample.ts
  blur.ts
  rotate.ts
  perspective.ts
  jpeg.ts
  runner.ts
forensics/
  geometry.ts
  moduleStats.ts
  structureScore.ts
  anomalyMap.ts
context/
  gmail.ts
  contextSignals.ts
destination/
  normalize.ts
  heuristics.ts
  reputationClient.ts
scoring/
  components.ts
  score.ts
  policy.ts
storage/
  chromeStorage.ts
  indexedDb.ts
  cache.ts
privacy/
  mode.ts
  redaction.ts
ui/
  components/...
  screens/...
  styles/...
fixtures/
  manifest.json
  stable/
  dual-module/
  low-quality/
public/
  icons/
  screenshots/
tests/
  unit/
  e2e/
api/                                # optional cloud layer
  src/
    routes/
    services/
    middleware/
    index.ts
packages/
  shared/
    schemas.ts
    types.ts
    constants.ts
docs/
  architecture.md
  privacy.md

```

threat-model.md
package.json
pnpm-workspace.yaml
README.md

33. Recommended Tooling

Area	Choice	Why
Extension framework	WXT	Good MV3 ergonomics and multiple extension entrypoints.
UI	React + TypeScript	Fast iteration and componentized side panel.
Styling	Tailwind CSS + small CSS modules	Speed without forcing a visual system on every component.
State	Zustand or React context	Small state surface; avoid overengineering.
Local DB	IndexedDB via idb	Scan history/cache without backend dependency.
QR decoders	ZXing + jsQR + OpenCV adapter	Decoder diversity with independent code paths.
Image processing	Canvas / OffscreenCanvas + OpenCV.js	Browser-local transforms.
Tests	Vitest + Playwright	Unit + extension E2E.
Optional backend	Fastify/TypeScript	Small, explicit API surface.
Package manager	pnpm	Monorepo workspace support.

34. Implementation Decisions That Prevent Common Vibe-Coding Failures

- **Do not build a web dashboard first.** The demo must prove the browser protection loop.
- **Do not start with AI.** Get deterministic decoding, transforms, evidence, and UI working first.
- **Do not hardwire Gmail selectors everywhere.** Isolate all Gmail DOM logic behind an adapter.
- **Do not let the extension call arbitrary external APIs from the UI.** Centralize network requests in the service worker or backend.
- **Do not store base64 images in localStorage.** Use short-lived memory or IndexedDB only when retention is enabled.
- **Do not label "Stable" as "Safe."** Keep integrity and destination risk as separate axes.
- **Do not show a score alone.** Every alert must show the evidence that changed the decision.
- **Do not create fake benchmark claims.** All performance numbers in this PRD are engineering targets, not measured results.

35. User Flows

Flow	Steps	Outcome
Safe QR	Gmail opens -> QR detected -> quick verify -> decoders agree -> badge Stable	User sees stable state; can open report or proceed.

Suspicious QR	QR detected -> quick verify detects uncertainty -> deep verify -> evidence matrix -> Block	User gets actionable warning before scanning.
Manual scan	Select image -> context menu "Verify with QRift" -> side panel -> report	Works even if automatic candidate detection missed.
False positive	Open report -> Mark trusted -> choose scope -> local allowlist	User can reduce friction without global disable.
Research demo	Research Lab -> choose controlled fixture -> Run Deep Verify -> evidence -> export JSON	Repeatable competition demo and regression test.

36. Acceptance Criteria

- 1. Install:** Extension loads in unpacked Chrome without manifest or service-worker errors.
- 2. Gmail:** On a local test Gmail-like page, QRift detects QR candidates and adds a non-blocking status badge.
- 3. Quick verify:** A clean QR produces a consistent payload across available decoders or explicitly reports unavailable decoders.
- 4. Deep verify:** The report contains decoder table, transform matrix, structural signals, destination, and context.
- 5. Instability:** A controlled dual-module fixture used in testing triggers at least one meaningful evidence signal under the documented environment.
- 6. Privacy:** Local-only mode performs no image upload.
- 7. Block UI:** Critical finding produces a clear block recommendation and never auto-opens the destination.
- 8. Performance:** Median quick and deep targets are measured on the reference laptop and reported honestly.
- 9. Tests:** Unit suite and Playwright smoke tests pass.
- 10. Demo:** Two-click switching between stable and controlled attack fixtures works reliably.

37. Definition of Done for the Vibe-Coded Build

- Repository runs from a clean checkout with one documented install command.
- Extension can be loaded in Chrome and side panel opens.
- Gmail test page and general test page both work.
- All core TypeScript types compile with no `any` in core engine paths.
- At least three decoder adapters are represented; capability gaps are handled gracefully.
- At least eight metamorphic transforms are implemented or explicitly feature-flagged.
- Score calculation is deterministic and unit-tested.
- UI states include loading, stable, review, critical, error, empty.
- Research fixtures ship with ground truth and no live malicious destinations.
- README includes architecture, permissions, local setup, demo steps, and privacy model.
- No unsupported security claims appear in UI copy or README.

38. Master Vibe-Coding Prompt

Paste the following as the primary build instruction to the coding agent. Keep this prompt intact; it is the condensed execution contract derived from the full PRD above.

You are building QRift, a Chrome Manifest V3 security extension. Build the complete repository in one implementation pass. Do not create a static mockup or a fake demo. The app must run locally and demonstrate the full user loop.

PRODUCT: QRift - "One QR. One meaning. Or don't trust it."

PRIMARY SURFACE: Chrome side panel + inline Gmail QR status badge.

CORE THESIS: QR security should validate interpretation stability, not only decode one URL.

TECH STACK

- WXT + React + TypeScript
- Tailwind CSS for UI
- IndexedDB (idb) for local history/cache
- Chrome APIs: sidePanel, storage, scripting, activeTab, optional contextMenus
- Gmail host permission only for automatic Gmail mode; optional broader host permissions for user-enabled web mode
- Vitest + Playwright
- Decoder adapters: ZXing, jsQR, OpenCV QRCodeDetector. Isolate all decoder dependencies behind a common interface.
- Image transforms: Canvas/OffscreenCanvas/OpenCV.js

PRIMARY FEATURES

1. Candidate QR detection on Gmail/web pages using throttled MutationObserver + IntersectionObserver. Do not scan every image continuously.
2. Quick Verify: run available decoders, basic structure checks, URL normalization. Cache result.
3. Deep Verify: multi-decoder comparison + structural forensics + metamorphic transform suite + context analysis + destination analysis.
4. Interpretation Stability Score from explainable components. Store all raw component metrics. Default heuristic weights: 0.40 decoder consensus, 0.30 transform consistency, 0.20 structure, 0.10 context. Thresholds: 85 stable, 70 review, 40 unstable, below 40 critical. Treat these as configurable MVP heuristics, not validated science.
5. If decoders return different payloads, label DECODER_MISMATCH and escalate to critical/block according to policy. Distinguish decoder failure from disagreement.
6. Metamorphic suite: scale, resample, blur, rotate, perspective, JPEG, contrast, print-scan approximation. Each test records condition, result, payload hash, elapsed time.
7. Structural forensics: perspective-correct QR, estimate grid, compute module mean luminance/center luminance/variance/edge density, flag repeated internal contrast, check finder/timing/alignment/quiet zone heuristics, generate anomaly heatmap data. Never use one heuristic as a malicious verdict.
8. Context: compare email/page claim, sender domain, visible links, and QR hostname. Explain mismatches in plain language.
9. Destination: normalize URL, display hostname prominently, optional safe reputation adapter. Never embed secret provider keys in extension.
10. Local-first privacy. Raw QR images are never persisted by default. Cloud analysis must be opt-in and visibly indicated.
11. Research Lab route for controlled fixtures. Include stable QR, dual-module controlled fixture, low-quality QR, and benign complex QR. Use example.com / example.invalid style reserved domains and do not ship live phishing URLs.
12. Research-grade evidence report with: verdict, score, decoder matrix, transform matrix, structural findings, destination, context, action.

UI REQUIREMENTS

- Side panel is the main workspace, dark graphite with slate cards and restrained cyan accents.
- No glassmorphism, no rainbow gradients, no over-designed AI aesthetic.
- Screens: Home, QR Detail, Deep Verify, Settings, Research Lab.
- Components: AppShell, Header, PageSummary, QRCard, QRThumbnail, IntegrityScore, VerdictBanner, EvidenceRow, DecoderTable, TransformMatrix, DestinationPanel, ContextPanel, ActionBar, PrivacyBadge, EmptyState, ErrorState.
- Stable: green check + "Stable under tested conditions".
- Review: amber + one-line reason.
- Critical: red + "Interpretation unstable" + "Do not open or scan this QR".
- Never write "Safe" solely because QR is stable; integrity and destination risk are separate.
- Show score and reasons together.

EXTENSION ARCHITECTURE

- background.ts: orchestration, message routing, side panel opening, cache, policy.
- content.ts: Gmail/web candidate detection and inline badges.
- sidepanel.tsx: React UI.
- offscreen.ts: heavy image processing and decoder execution.
- core pipeline: quick/deep verification orchestration.
- messages: typed request/response discriminated unions.

DATA TYPES

- QRArtifact
- DecodeResult
- StructureReport
- TransformResult
- DestinationReport
- ContextReport
- Finding
- ScanReport
- Policy

SECURITY

- Do not auto-open QR destinations.
- Never browse untrusted URLs invisibly from the extension.
- Sanitize all page-derived text before rendering.
- Rate limit cloud API.
- Do not request all-web permissions by default.
- No remote executable code.

TESTING

- Unit tests for URL normalization, payload grouping, score calculation, structure metrics.
- Fixture tests for all decoder adapters.
- Transform regression tests.
- Playwright test page with clean and controlled suspicious fixtures.
- Extension smoke test: detect -> verify -> report -> block.

DELIVERABLES

- Fully working repository.
- README with install, architecture, permissions, demo flow, privacy, and troubleshooting.
- Seed fixtures and ground-truth manifest.
- Playwright smoke tests.
- Clean TypeScript build.
- No TODO placeholders in the primary user flow.
- No fake benchmark claims.

IMPLEMENTATION ORDER

Do the work in dependency order inside the single build operation: scaffold extension -> shared types -> decoder adapters -> image pipeline -> structural forensics -> transforms -> scoring/policy -> storage -> side panel UI -> Gmail integration -> research lab -> tests -> README. Do not stop after scaffolding. Run tests/build and fix errors before completion.

39. Demo Script for Judges

- 1. Baseline:**Open a normal QR email. QRift badge says Stable. Side panel shows decoder agreement and high integrity.
- 2. Attack fixture:**Switch to the controlled dual-module fixture. Standard decode view appears benign/expected, then QRift deep verification shows structural anomaly and/or decoder divergence in the tested configuration.
- 3. Evidence:**Open Deep Verify. Point to the interpretation comparison and transform matrix. This is the technical moment.
- 4. Action:**QRift recommends BLOCK and never opens the destination.
- 5. Research link:**Explain that the assigned paper showed the security problem can arise because QR scanners assume a single layer and use predictable decoding; QRift operationalizes the paper's recommended defensive direction: structure + visual/behavioral testing + context.

40. Competition Pitch Spine

Slide idea	What to say
Problem	A QR hides its destination from humans. A scanner can say "decoded" without proving the image itself is trustworthy.
Research insight	Dual-module QR demonstrates that one visual artifact can carry layered information that different processing paths may interpret differently.
Product	QRift is a browser-side QR integrity firewall.
Technical moat	Decoder diversity + metamorphic stress testing + module-level forensics + explainable evidence.
Demo	Normal QR passes. Controlled dual-module fixture produces instability evidence. QRift blocks before the user scans.
Market wedge	Gmail/browser protection for users and security teams; later API/SDK for enterprise security surfaces.
Why now	Microsoft reported QR-phishing volumes rising from 7.6M in Jan 2026 to 18.7M in Mar 2026, a 146% increase in Q1 2026. [WEB: Microsoft Security Blog, Apr 30 2026]

41. Competitive Reality Check

The market already contains QR-aware email security and browser extensions. INKY publicly lists QR code detection plus computer vision in its email security platform; Microsoft Defender has published QR-phishing defenses; browser extensions such as clearQR and PhiShark already offer QR scanning and URL-risk analysis. QRift therefore cannot win by claiming merely "AI-powered QR phishing detection."

Competitor category	Observed capability	QRift response
Enterprise email security	QR decode + URL investigation; computer vision/AI.	Focus product on QR interpretation integrity and browser-side evidence.
Generic browser QR scanners	Scan page QRs, inspect destination, URL risk.	Do not compete on scanning convenience; compete on stability analysis.
Phishing browser extensions	Real-time URL/page checks, QR/PDF risk.	Own the QR artifact integrity layer and research-grade explanation.
Open-source decoders	Fast QR extraction.	Use multiple decoders as sensors; never pretend decoding equals security.

Positioning sentence

"Most tools ask whether the destination is bad. QRift asks whether the QR is lying about what it means." This is the core narrative - but it must be backed by observed decoder / transform evidence in every suspicious demo.

42. Future Expansion (Not Required for MVP)

- **Enterprise browser policy:** Chrome Enterprise policy deployment, central policy and reporting.
- **PDF / DOCX scanning:** analyze QR images embedded in office documents, directly aligned with the paper’s mixed-media future research direction.

- **Mobile SDK:** real camera pipeline with device-specific calibration instead of browser-only simulation.
- **Trust certificates:** organization-signed QR integrity certificates for pre-registered QR assets; only after core detection is validated.
- **Visual AI model:** learn structural anomaly patterns from the research corpus to supplement deterministic checks.
- **New QR formats:** FrameQR / Micro QR and other formats noted as future research in the paper.

43. Non-Goals for the Competition Build

- Not a full email security gateway replacement.
- Not a universal phishing detector for every web threat.
- Not a real-time malware sandbox.
- Not an exact emulator of every phone camera.
- Not a blockchain certificate product in the first release.
- Not a claim that every QR manipulation produces a second payload.

44. Source and Evidence Notes

The product requirements above are grounded first in the assigned paper and then expanded with current Chrome extension platform documentation and current market observations. Where an engineering target or proposed feature is not established by the source paper, it is explicitly framed here as a product design decision, target, or hypothesis rather than a proven fact.

Source	Use in PRD
Assigned paper: Steve Legere, "Dual-Module QR Codes: Bypassing QR Code Scanners in Enterprise Email Gateways" (accepted June 1, 2026).	Threat model, dual-module behavior, experimental conditions, findings, defense recommendations, future research.
Chrome for Developers - Extensions API reference.	Side panel, offscreen, storage, scripting, permissions architecture.
Chrome for Developers - Permissions.	Least-privilege permission strategy.
Microsoft Security Blog, Apr. 30, 2026.	Current market threat-growth context: 7.6M -> 18.7M QR-phishing attacks in Q1 2026.
INKY Email Security Platform / QR Code + Computer Vision pages.	Competitive reality: QR detection and computer vision already exist in email security.
Chrome Web Store: clearQR / PhiShark.	Competitive reality: browser extensions already perform QR and phishing analysis.

45. References

- [PAPER] Legere, Steve. "Dual-Module QR Codes: Bypassing QR Code Scanners in Enterprise Email Gateways." Accepted June 1, 2026. Internal assigned source. Key pages cited: 9-10, 13-20, 21-23.
- [WEB-1] Chrome for Developers. Extensions API reference.
<https://developer.chrome.com/docs/extensions/reference/api>
- [WEB-2] Chrome for Developers. Permissions.
<https://developer.chrome.com/docs/extensions/reference/permissions-list>

- [WEB-3] Microsoft Security Blog. "Email threat landscape: Q1 2026 trends and insights." Apr. 30, 2026. <https://www.microsoft.com/en-us/security/blog/2026/04/30/email-threat-landscape-q1-2026-trends-and-insights/>
- [WEB-4] INKY. "Email Security Platform" and "Combating Malicious QR Codes with Computer Vision." <https://www.inky.com/email-security-platform> ; <https://www.inky.com/en/blog/combating-malicious-qr-codes-with-computer-vision>
- [WEB-5] Chrome Web Store. clearQR - Secure QR Scanner. <https://chromewebstore.google.com/detail/clearqr-secure-qr-scanner/hfpehjennkkajefphhkmbfamfjehhpok>
- [WEB-6] Chrome Web Store. PhiShark - Browser Phishing & Link Protection. <https://chromewebstore.google.com/detail/phishark-browser-phishing/mfegmmnkbalapbgilfchfdmkleopddhl>
- [WEB-7] Microsoft Security Blog. "How Microsoft Defender for Office 365 innovated to address QR code phishing attacks." Nov. 4, 2024. <https://www.microsoft.com/en-us/security/blog/2024/11/04/how-microsoft-defender-for-office-365-innovated-to-address-qr-code-phishing-attacks/>

Appendix A - Research Anchors from the Assigned Paper

Paper page	Anchor
2-4	QR adoption, quishing threat surface, and human opacity of QR contents.
9-10	Predictable decoding and common libraries; research gap and structural-analysis direction.
13-14	Experiment setup and 99.87% +/- 0.1277% bypass result under the tested Sophos configuration.
15-17	Version 6 practical limit in the tested attack, sub-module size behavior, HTML resize behavior, mobile decoding.
18	Table of tested bypass outcomes by sub-module size.
19-20	Defense: standards/conformance, average-vs-center module analysis, AI pattern recognition, context manipulation tests.
21-23	Future research: parameter mapping, alternate formats, mixed media, visual AI, context simulation, LLM-enabled analysis.

Appendix B - Exact Product Copy

UI location	Copy
Stable badge	QRift: Stable under tested conditions
Review badge	QRift: Needs review
Critical badge	QRift: Interpretation unstable
Critical action	Do not open or scan this QR
Decoder mismatch	Different decoding paths produced different results.
Transform finding	Payload changed under a rendering stress test.
Structure finding	QR modules show unusual internal contrast.
Context finding	QR destination does not match the surrounding message context.
Privacy badge	Local analysis - image stays on this device

Incomplete	Verification incomplete - this does not mean safe.

Appendix C - Final Build Checklist

Check	Pass condition
Extension install	Loads with no manifest/background errors.
Side panel	Opens and lists QR artifacts.
Gmail badge	Appears next to candidate QR and is re-anchored after SPA DOM changes.
Quick verify	Returns a typed result and cache entry.
Deep verify	Shows all evidence modules.
Dual-module fixture	Produces observed instability evidence in documented environment.
Local mode	No image uploads.
Tests	Vitest + Playwright pass.
Build	Production bundle generated.
Demo	Stable vs suspicious story completes in under 3 minutes.